

**Evaluate and analyze various page replacement strategies to optimize memory management and enhance system performance in operating systems.**

### *Objectives*

- To validate the three-page replacement algorithms, first in first out, optimal page replacement strategy and least recently used replacement algorithm.

### *Task1(a)*

*Comment the given code for FIFO page replacement*

```
#include <stdio.h>

// function for FIFO page replacement algorithm
void FIFO(int pages[], int n, int capacity)
{
    int frames[capacity]; // array to store pages in frames
    int front = 0; // // Points to the next frame to be replaced
(FIFO)
    int pageFaults = 0; // count of page faults

    // Initialize all frames to -1 indicating they are empty
    for (int i = 0; i < capacity; i++)
        frames[i] = -1;
    // Process each page in the reference string
    for (int i = 0; i < n; i++)
    {
        // Flag to check if page is found in frames, initially set
to 0 (not found)
        int found = 0;
        // Check if the current page is already in one of the
frames
        for (int j = 0; j < capacity; j++)
        {
            // If page is found in frames, set found to 1 and
```

## LAB # 10 Page replacement strategies

---

```
break
    if (frames[j] == pages[i])
    {
        found = 1;
        break;
    }
}

// If page is not found in frames, replace the oldest
page (FIFO)
if (!found)
{
    frames[front] = pages[i];
    front = (front + 1) % capacity;
    pageFaults++;
}
}

printf("FIFO Page Faults: %d\n", pageFaults);
}

int main(void)
{
    int pages[] = { 1, 3, 0, 3, 5, 6, 3 };
    int n = sizeof(pages) / sizeof(pages[0]);
    int capacity = 3;

    FIFO(pages, n, capacity);
    return 0;
}
```

*Brief the working/output:*

```
muhammad-ahmad@latitude-7490:~/os/lab10$ ./task1.o
FIFO Page Faults: 6
```

The algorithm processes each page in the reference string (pages[]):

## LAB # 10 Page replacement strategies

---

Checks if the page is already in frames[].  
If not found (page fault), replaces the page at front with the new page, increments front (circularly), and counts the fault.

Task 1(b)

Edit the code to print out hit and miss ratio and show the output for frame size 3.

```
#include <stdio.h>

void FIFO(int pages[], int n, int capacity)
{
    int frames[capacity];
    int front = 0;
    int pageFaults = 0;
    int hits = 0;

    for (int i = 0; i < capacity; i++)
        frames[i] = -1;
    for (int i = 0; i < n; i++)
    {
        int found = 0;
        for (int j = 0; j < capacity; j++)
        {
            if (frames[j] == pages[i])
            {
                found = 1;
                break;
            }
        }

        if (!found)
        {
            frames[front] = pages[i];
            front = (front + 1) % capacity;
            pageFaults++;
        }
        else
    }
```

## LAB # 10 Page replacement strategies

---

```
        {
            hits++;
        }
    }

    printf("FIFO Page Faults: %d\n", pageFaults);
    printf("Hits: %d\n", hits);
    printf("Hit Ratio: %.2f\n", (float)hits / n);
    printf("Miss Ratio: %.2f\n", (float)pageFaults / n);
}

int main(void)
{
    int pages[] = { 1, 3, 0, 3, 5, 6, 3 };
    int n = sizeof(pages) / sizeof(pages[0]);
    int capacity = 3;

    FIFO(pages, n, capacity);
    return 0;
}
```

*Brief the working/output:*

```
muhammad-ahmad@latitude-7490:~/os/lab10$ ./task2.o
FIFO Page Faults: 6
Hits: 1
Hit Ratio: 0.14
Miss Ratio: 0.86
```

*Task1(c)*

*Increase the frame size to 4 and observe the output with hit ratio and miss ratio, what results do you observe with the increase in frame size.*

*By increasing the frame size the hit ratio increases and the miss ratio decreases*

```
muhammad-ahmad@latitude-
7490:~/os/lab10$ ./task2.o
```

## LAB # 10 Page replacement strategies

---

FIFO Page Faults: 5  
Hits: 2  
Hit Ratio: 0.29  
Miss Ratio: 0.71

Task2(a)

*Comment the given code for least recently used page replacement.*

```
#include <stdio.h>

// Function for LRU page replacement algorithm
void LRU(int pages[], int n, int capacity)
{
    int frames[capacity]; // Array to store pages in frames
    int time[capacity];   // Array to track the last access time
    of each frame
    int pageFaults = 0;    // Count of page faults
    int t = 0;            // Global time counter

    // Initialize all frames to -1 indicating they are empty
    for (int i = 0; i < capacity; i++)
        frames[i] = -1;

    // Process each page in the reference string
    for (int i = 0; i < n; i++)
    {
        int found = 0;      // Flag to check if page is found in
frames
        int least = 0;      // Index of the least recently used
frame

        // Check if the current page is already in one of the
frames
        for (int j = 0; j < capacity; j++)
        {
            if (frames[j] == pages[i])
            {
                found = 1;
                time[j] = ++t; // Update access time
                break;
            }
        }
    }
}
```

## LAB # 10 Page replacement strategies

```
    // If page is not found, replace the least recently used
page
    if (!found)
    {
        // Find the frame with the smallest time (least
recently used)
        for (int j = 1; j < capacity; j++)
        {
            if (time[j] < time[least])
                least = j;
        }
        frames[least] = pages[i];
        time[least] = ++t;
        pageFaults++;
    }
}

printf("LRU Page Faults: %d\n", pageFaults);
}

int main()
{
    int pages[] = {1, 3, 0, 3, 5, 6, 3};
    int n = sizeof(pages) / sizeof(pages[0]);
    int capacity = 3;

    LRU(pages, n, capacity);
    return 0;
}
```

*Brief the working/output:*

```
muhammad-ahmad@latitude-7490:~/os/lab10$ ./task2.o
LRU Page Faults: 7
```

Initialize frames[] to -1 (empty) and time[] to 0.

1. For each page in the reference string:

- Check if the page is in frames[].
- If found (hit): Update its time[j] to the current t (incremented).
- If not found (miss): Find the frame with the smallest time[] value (least recently used). Replace it with the new page, update its time[], and increment pageFaults.

## LAB # 10 Page replacement strategies

---

2. Print the total page faults.

*Task2(b)*

*Edit the code to print hit and miss ratio also Execute the code using the same page number string used In FIFO for frame size 3.*

```
if (!found)
{
    // Find the frame with the smallest time (least
recently used)
    for (int j = 1; j < capacity; j++)
    {
        if (time[j] < time[least])
            least = j;
    }
    frames[least] = pages[i];
    time[least] = ++t;
    pageFaults++;
}
else
{
    hits++;
}
}
```

*Brief the working/output:*

```
muhammad-ahmad@latitude-7490:~/os/lab10$ ./task2.o
LRU Page Faults: 7
Hits: 0
Hit Ratio: 0.00
Miss Ratio: 1.00
```

## LAB # 10 Page replacement strategies

---

### Task2(c)

*Edit the code to except the page string from command line with dynamic length.*

*Brief the working/output:*

```
muhammad-ahmad@latitude-7490:~/os/lab10$ ./task2a.o
Enter the number of pages: 4
Enter the page sequence: 1 3 0 3 5 6 3
LRU Page Faults: 3
Hits: 1
Hit Ratio: 0.25
Miss Ratio: 0.75
```

### Task3(a)

*comment the given code for Optimal page replacement*

```
#include <stdio.h>

// Function to implement Optimal Page Replacement
void Optimal(int pages[], int n, int capacity)
{
    int frames[capacity], pageFaults = 0;

    // Initialize all frames to -1 (empty)
    for (int i = 0; i < capacity; i++)
        frames[i] = -1;

    // Traverse each page request
    for (int i = 0; i < n; i++)
    {
        int found = 0;

        // Check if page already exists in any frame
        for (int j = 0; j < capacity; j++)
```



## LAB # 10 Page replacement strategies

---

```
{
    if (frames[j] == pages[i])
    {
        found = 1;    // Page Hit
        break;
    }
}

// If page is not found → Page Fault
if (!found)
{
    int farthest = -1, replace = -1;

    // Check future use of each frame page
    for (int j = 0; j < capacity; j++)
    {
        int k;

        // Scan ahead to find next occurrence
        for (k = i + 1; k < n; k++)
        {
            if (frames[j] == pages[k])
                break;
        }

        /*
        If k goes beyond the future requests,
        it means this page is *never used again* →
best to replace.
        */
        if (k > farthest)
        {
            farthest = k;
            replace = j;
        }
    }

    // Replace the page that will not be used for the
longest time
    frames[replace == -1 ? 0 : replace] = pages[i];
}
```

## LAB # 10 Page replacement strategies

---

```
        pageFaults++;
    }
}

// Print total page faults
printf("Optimal Page Faults: %d\n", pageFaults);
}

int main()
{
    int pages[] = {1, 3, 0, 3, 5, 6, 3};
    int n = sizeof(pages) / sizeof(pages[0]);
    int capacity = 3;

    // Call Optimal algorithm
    Optimal(pages, n, capacity);

    return 0;
}
```

*Brief the working/output:*

Optimal Page Faults: 5

Start: Set all frames[] = -1 (empty).

For each page:

If page is already in frames then hit.

If not found then miss:

Look ahead in future references.

Find the page that will be used farthest in the future (or never again).

Replace that page in frames[].

Count and print total page faults.

*Task3(b)*

*Edit the code to print hit and miss ratio also Execute the code and compare the Efficiency in terms of Hit Ratio and Miss Ratio with LRU and FIFO with same page reference String.*

*Output:*

## LAB # 10 Page replacement strategies

---

```
--- Optimal Page  
Replacement ---  
Total Hits: 2  
Total Misses: 5  
Hit Ratio: 0.286  
Miss Ratio: 0.714
```

Task 3 (c) Comparison Performance FIFO, LRU, Optimal

```
#include <stdio.h>

void FIFO(int pages[], int n, int capacity)
{
    int frames[capacity];
    int index = 0, hits = 0, faults = 0;

    for (int i = 0; i < capacity; i++)
        frames[i] = -1;

    for (int i = 0; i < n; i++)
    {
        int found = 0;
        for (int j = 0; j < capacity; j++)
        {
            if (frames[j] == pages[i])
            {
                hits++;
                found = 1;
                break;
            }
        }

        if (!found)
        {
            frames[index] = pages[i];
            index = (index + 1) % capacity;
            faults++;
        }
    }

    printf("\nFIFO\nHits=%d Misses=%d Hit Ratio=%.2f Miss  
Ratio=%.2f",
           hits, faults, (float)hits/n, (float)faults/n);
}
```

## LAB # 10 Page replacement strategies

---

```
}

void LRU(int pages[], int n, int capacity)
{
    int frames[capacity], recent[capacity];
    int hits = 0, faults = 0;

    for (int i = 0; i < capacity; i++)
        frames[i] = recent[i] = -1;

    for (int i = 0; i < n; i++)
    {
        int found = 0;
        for (int j = 0; j < capacity; j++)
        {
            if (frames[j] == pages[i])
            {
                hits++;
                recent[j] = i;
                found = 1;
                break;
            }
        }

        if (!found)
        {
            int lru = 0;
            for (int j = 1; j < capacity; j++)
                if (recent[j] < recent[lru])
                    lru = j;

            frames[lru] = pages[i];
            recent[lru] = i;
            faults++;
        }
    }

    printf("\nLRU\nHits=%d Misses=%d Hit Ratio=%.2f Miss\n",
           hits, faults, (float)hits/n, (float)faults/n);
}

void Optimal(int pages[], int n, int capacity)
```

## LAB # 10 Page replacement strategies

---

```
{
    int frames[capacity];
    int hits = 0, faults = 0;

    for (int i = 0; i < capacity; i++)
        frames[i] = -1;

    for (int i = 0; i < n; i++)
    {
        int found = 0;
        for (int j = 0; j < capacity; j++)
        {
            if (frames[j] == pages[i])
            {
                hits++;
                found = 1;
                break;
            }
        }

        if (!found)
        {
            int farthest = -1, replace = 0;
            for (int j = 0; j < capacity; j++)
            {
                int k;
                for (k = i + 1; k < n; k++)
                    if (frames[j] == pages[k])
                        break;

                if (k > farthest)
                {
                    farthest = k;
                    replace = j;
                }
            }
            frames[replace] = pages[i];
            faults++;
        }
    }

    printf("\nOptimal\nHits=%d Misses=%d Hit Ratio=%.2f Miss
Ratio=%.2f",
```

## LAB # 10 Page replacement strategies

---

```
        hits, faults, (float)hits/n, (float)faults/n);
}

int main()
{
    int pages[] = {1, 3, 0, 3, 5, 6, 3};
    int n = sizeof(pages) / sizeof(pages[0]);
    int capacity = 4;

    FIFO(pages, n, capacity);
    LRU(pages, n, capacity);
    Optimal(pages, n, capacity);

    return 0;
}
```

```
muhammad-ahmad@latitude-7490:~/os/lab10$ ./task3b.o
```

*FIFO*

*Hits=1 Misses=6 Hit Ratio=0.14 Miss Ratio=0.86*

*LRU*

*Hits=2 Misses=5 Hit Ratio=0.29 Miss Ratio=0.71*

*Optimal*

*Hits=2 Misses=5 Hit Ratio=0.29 Miss Ratio=0.71*

*Task3(c)*

*Execute the code using the same page number string used In FIFO for frame size 4*

*Brief the working/output:*

```
muhammad-ahmad@latitude-7490:~/os/lab10$ ./task3b.o
```

*FIFO*

*Hits=2 Misses=5 Hit Ratio=0.29 Miss Ratio=0.71*

*LRU*

*Hits=2 Misses=5 Hit Ratio=0.29 Miss Ratio=0.71*

*Optimal*

*Hits=2 Misses=5 Hit Ratio=0.29 Miss Ratio=0.71*

## LAB # 10 Page replacement strategies

---

Table 1: Revise statements of LAB rubric for assessment

| The student performance for the assigned task during the lab session was: |                                                                                                                                                              |   |  |
|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---|--|
| Excellent                                                                 | The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately. | 4 |  |
| Good                                                                      | The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.        | 3 |  |
| Average                                                                   | The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.           | 2 |  |
| Worst                                                                     | The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.                          | 1 |  |